FRONT END DESIGN ENGINEERING

PROVA SEMESTRAL - DP 2026 - 2o SEMESTRE

VITE + REACT + TYPESCRIPT

Modelo de 10/11/2026 • Data: 10/11/2026 • Duração: 2 horas • Valor: 100 pontos

Instruções gerais para o aluno

- Tempo de prova: 2 horas. Leia as dez questões antes de iniciar e distribua o tempo entre implementação e conferência.
- Objetivo: completar o projeto TechStore fornecido, aplicando componentes, props, dados locais, Hooks, rotas e Tailwind básico.
- Consulta permitida: apostilas disponíveis no Portal do Aluno e anotações pessoais manuscritas. VS Code e terminal local estão liberados.
- Não é permitido acessar a internet, usar ferramentas de IA, consultar serviços externos ou fazer push para repositório remoto.
- O pacote traz as dependências completas, dados, imagens locais e repositório Git iniciado. Em computadores Windows, execute install-offline.cmd uma vez para ajustar as dependências da plataforma, sem internet; depois inicie com npm run dev. Não execute npm install ou npm update.
- A entrega é feita pelo Portal do Aluno, na tarefa que será indicada pelo professor durante a prova. Envie a pasta compactada em .zip com o projeto completo e a pasta .git.

Projeto e estrutura obrigatória

A aplicação apresenta a TechStore, um catálogo de produtos locais. Não crie backend. O JSON e as imagens fornecidos não devem ser removidos nem substituídos por recursos externos.

```
tech-store/
├── public/
│   ├── data/produtos.json              (fornecido)
│   └── images/                         (ilustrações locais fornecidas)
├── src/
│   ├── components/CardProduto.tsx      (Questão 3)
│   ├── routes/
│   │   ├── Home/index.tsx              (fornecido)
│   │   ├── Produtos/index.tsx          (Questões 2 e 4–7)
│   │   └── ProdutoDetalhes/index.tsx   (Questão 8)
│   ├── services/produtos.ts            (Questão 1)
│   ├── types/produto.ts                (fornecido)
│   ├── App.tsx e main.tsx              (layout e rotas fornecidos)
│   └── index.css                       (Tailwind configurado)
├── package.json / package-lock.json
└── README_ALUNO.md
```

Questões práticas

Questão 1 — Leitura do JSON local — 10 pontos

Em src/services/produtos.ts, implemente buscarProdutos usando fetch para /data/produtos.json. Verifique resposta.ok; em caso de falha, interrompa o fluxo com um erro. Retorne a lista como Produto[].

Questão 2 — Carregamento e erro — 10 pontos

Em src/routes/Produtos/index.tsx, use useEffect para chamar buscarProdutos quando a página montar. Atualize produtos e carregando. Se ocorrer falha, mostre uma mensagem de erro visível e encerre o estado de carregamento.

Questão 3 — Card tipado — 10 pontos

Complete src/components/CardProduto.tsx. Receba um Produto por props e exiba imagem, nome, categoria, status e preço. A imagem deve ter texto alternativo ligado ao nome do produto.

Questão 4 — Lista de produtos — 10 pontos

Na página Produtos, renderize um CardProduto para cada produto usando map. Passe o produto por props e use produto.id como key.

Questão 5 — Busca controlada — 10 pontos

Inclua um campo de busca controlado por useState. A busca deve ignorar maiúsculas/minúsculas e procurar pelo nome do produto.

Questão 6 — Filtro controlado — 10 pontos

Inclua um select controlado por useState com uma opção Todos e os valores existentes no JSON. Filtre por categoria: notebook, smartphone ou acessorio.

Questão 7 — Combinação de resultados — 10 pontos

A busca e o select devem funcionar ao mesmo tempo. Calcule a lista visível a partir dos produtos, da busca e do filtro, sem criar state para uma lista derivada. Quando não houver correspondências, mostre “Nenhum produto encontrado”.

Questão 8 — Detalhes pela URL — 10 pontos

Complete src/routes/ProdutoDetalhes/index.tsx para ler :id com useParams, localizar o produto e exibir imagem, nome, descrição, categoria, status e preço. Mostre “Produto não encontrado” para um id inexistente e mantenha o link de retorno ao catálogo.

Questão 9 — Apresentação mínima — 10 pontos

Use Tailwind para organizar o catálogo em uma grade responsiva (uma coluna em tela estreita, duas a partir do breakpoint médio e três em tela larga). Mantenha rótulos visíveis nos controles e foco perceptível ao navegar pelo teclado.

Questão 10 — Git local — 10 pontos

Além do commit inicial fornecido, registre pelo menos dez novos commits durante a prova, em etapas diferentes, com mensagens claras. Confira o histórico com git log --oneline e não faça push.

Pontuação

#	Critério	O que será observado	Pontos
1	JSON local	Função assíncrona, caminho correto, verificação de resposta e retorno tipado.	10
2	Carregamento e erro	Effect na montagem, atualização dos estados e mensagens visíveis.	10
3	Card tipado	Props Produto e apresentação dos campos e imagem com alt.	10
4	Renderização da lista	map, props corretas e key estável pelo id.	10
5	Busca	Controle por state e correspondência de texto conforme o modelo.	10
6	Filtro	Select controlado e opções/valores corretos para o modelo.	10
7	Combinação e vazio	Busca e filtro combinados, lista derivada e mensagem vazia.	10

#	Critério	O que será observado	Pontos
8	Detalhes	useParams, produto correto, campos, id não encontrado e retorno.	10
9	Tailwind e acessibilidade	Grade responsiva, labels e foco de teclado.	10
10	Git local	Dez novos commits distribuídos e histórico local verificável.	10

Entrega

- Ao final, verifique o projeto no navegador e confira se o terminal não apresenta erro que impeça a execução.
- Compacte o projeto em .zip com src, public, package.json, package-lock.json e .git. Não inclua node_modules, .npm-cache, dist ou pastas temporárias; esses arquivos aumentam o tamanho e podem ser reconstruídos pelo professor a partir do pacote-base.
- Envie o .zip na tarefa do Portal do Aluno informada pelo professor. Não envie por GitHub e não faça push.

Esta prova possui um único modelo para a data de 10/11/2026. As dez questões valem 10 pontos cada; total: 100 pontos.